

DSP in VLSI

Homework 5

Fast Fourier Transform (FFT)

Student ID

M11407439

Name

MING-HONG, LIN

Development Environment

HDL Verilog

Simulation Tools Synopsys VCS and Verdi

Synthesis Tool Synopsys DC Compiler

Process TSMC 16nm (ADFP)

Verification Python

Due Date: 2026/05/20

Contents

1	Step 1 — SDF FFT Implementation	2
1.1	Result 1 - SDF FFT Output in Bit-Reversed Order	2
2	Step 2 — Bit-Reversal Re-Ordering and Verification	3
2.1	Result 2 - Output Re-Ordering	3
2.2	Result 2 - Absolute Error Verification	4
3	Step 3 — Fixed-Point SQNR Analysis	5
3.1	Result 3 - SQNR versus Fractional Wordlength	5
4	Step 4 — Control Signals: Butterfly and Commutator	7
4.1	Result 4 - Control Signal Generation	7
5	Step 5 — Control Signals: Complex Multipliers	8
5.1	Result 5 - Multiplier Control Signal Generation	8
6	Step 6 — Twiddle Factor ROM Design	9
6.1	Result 6 - Lookup Table and Quadrant Symmetry	9
7	Step 7 — Bit-Reversal Ping-Pong Buffer	11
7.1	Result 7 - Timing Diagram of Bit-Reversal Module	11
8	Step 8 — SDF FFT Behavior Simulation	12
8.1	Result 8 - Single Symbol Simulation	12
9	Step 9 — Streaming Simulation (96 Samples)	13
9.1	Result 9 - Streaming-In Streaming-Out Verification	13
10	Step 10 — Synthesis Without Pipeline	14
10.1	Result 10 - Pre-Pipeline Synthesis Report	14
11	Step 11 — Pipeline Synthesis at 300 MHz	15
11.1	Result 11 - Post-Pipeline Synthesis and Simulation	15

Step 1 — SDF FFT Implementation

1.1 Result 1 - SDF FFT Output in Bit-Reversed Order

Please use `FFTInput32.mat` as the inputs. Use your SDF FFT Matlab/Python program to generate the 32 FFT outputs. Because they are in bit-reversed order, write a program to generate their associated frequency domain indices. Now, you have an 1×32 array containing the frequency domain indices and the SDF outputs. List the array. (10%)

Table 1.1: SDF FFT Output in Bit-Reversed Order (`FFTInput32.mat`)

Pos. k	Freq. Index	Real	Imag	Pos. k	Freq. Index	Real	Imag
0	$X[0]$	-1	$-j$	16	$X[1]$	-1	$-j$
1	$X[16]$	-1	$+j$	17	$X[17]$	-1	$+j$
2	$X[8]$	+1	$-j$	18	$X[9]$	+1	$+j$
3	$X[24]$	+1	$-j$	19	$X[25]$	-1	$+j$
4	$X[4]$	-1	$+j$	20	$X[5]$	+1	$+j$
5	$X[20]$	+1	$-j$	21	$X[21]$	+1	$-j$
6	$X[12]$	+1	$+j$	22	$X[13]$	+1	$+j$
7	$X[28]$	+1	$-j$	23	$X[29]$	-1	$+j$
8	$X[2]$	-1	$-j$	24	$X[3]$	+1	$-j$
9	$X[18]$	+1	$-j$	25	$X[19]$	+1	$-j$
10	$X[10]$	-1	$+j$	26	$X[11]$	-1	$+j$
11	$X[26]$	-1	$-j$	27	$X[27]$	+1	$-j$
12	$X[6]$	+1	$-j$	28	$X[7]$	+1	$+j$
13	$X[22]$	-1	$+j$	29	$X[23]$	-1	$-j$
14	$X[14]$	-1	$+j$	30	$X[15]$	+1	$-j$
15	$X[30]$	+1	$+j$	31	$X[31]$	+1	$+j$

Step 2 — Bit-Reversal Re-Ordering and Verification

2.1 Result 2 - Output Re-Ordering

Please use `FFTInput32.mat` as the inputs. Show that your program for output re-ordering is correct. Draw the real part and imaginary part of $X_0 \sim X_{31}$. (10%)

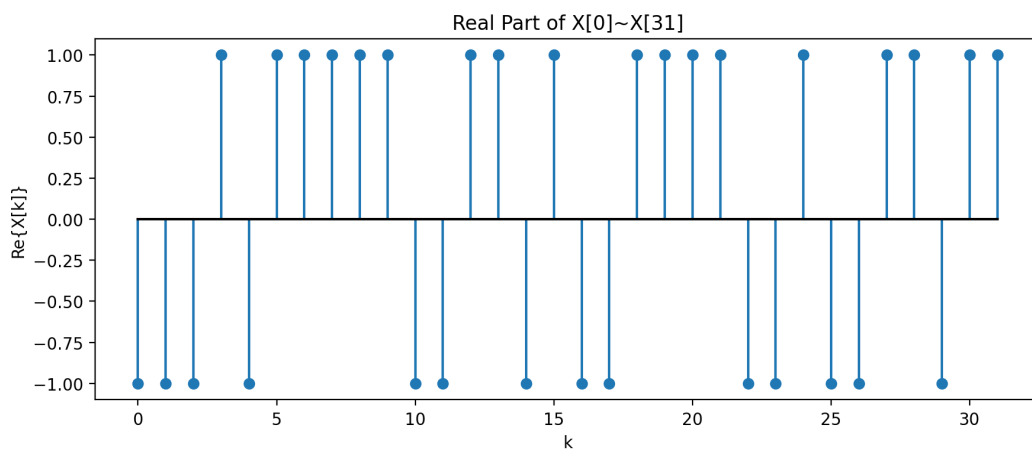


Figure 2.1: Real Parts of $X_0 \sim X_{31}$ after Bit-Reversal Re-Ordering

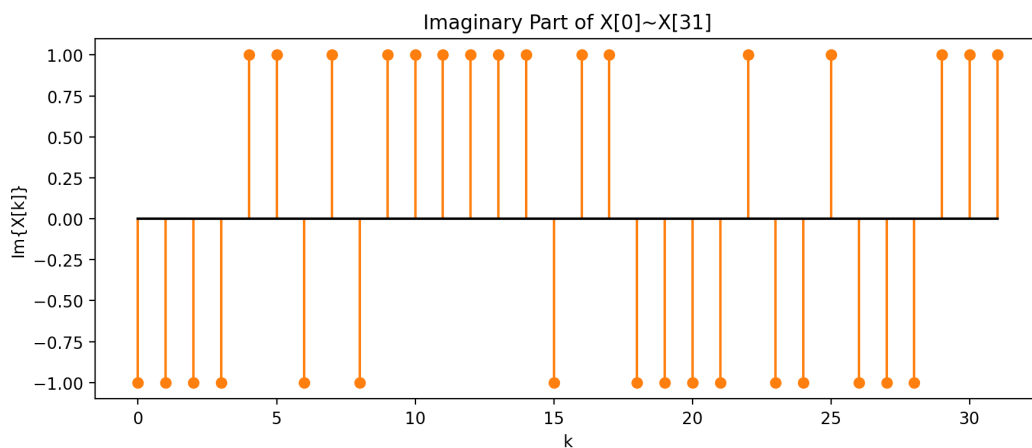


Figure 2.2: Imaginary Parts of $X_0 \sim X_{31}$ after Bit-Reversal Re-Ordering

2.2 Result 2 - Absolute Error Verification

Compare the results of $X_0 \sim X_{31}$ to $A_0 \sim A_{31}$. Draw the absolute error of each sample. The error should be smaller than 10^{-10} because of double-precision floating-point operations. If the error is not acceptable, the first 20 points will be deducted, too. (10%)

As shown in Figure 2.3, the maximum absolute error $|X_k - A_k|$ is below $7 \cdot 10^{-16}$ for all k .

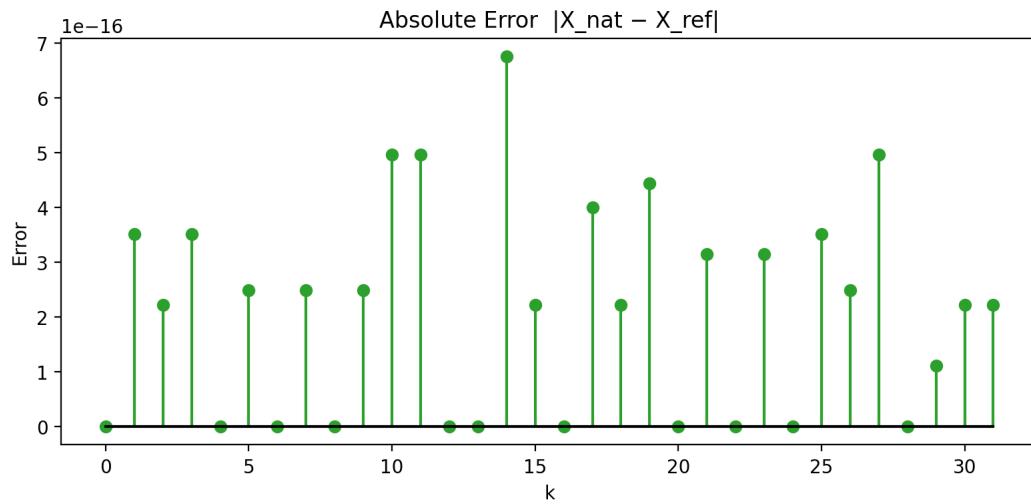


Figure 2.3: Absolute Error $|X_k - A_k|$ for $k = 0 \sim 31$

Step 3 — Fixed-Point SQNR Analysis

3.1 Result 3 - SQNR versus Fractional Wordlength

Generate your own inputs of 96 samples. Draw the SQNR versus fractional part word-length N stage by stage. (30%)

- Quantize the first stage. Evaluate the FFT output SQNR versus N for $N = 9, 10, \dots, 18$ for stage 1.
- Fix the setting for stage 1. Quantize the second stage. Draw the figure for stage 2.
- (d) Repeat for stages 3–5.
- Decide the same fractional part word-length for twiddle factors in the ROM tables of all stages. Draw the FFT output SQNR versus fractional part word-length of twiddle factors.

Per the instructor's update, the fractional part word-length N is swept from 7 to 19 for each stage and for the twiddle factors. As shown in the following figures, $N = 7$ is the first value that meets the **35 dB** SQNR requirement for every stage and for the twiddle ROM; therefore the selected fractional word-length is 7 for all stages and twiddle factors.

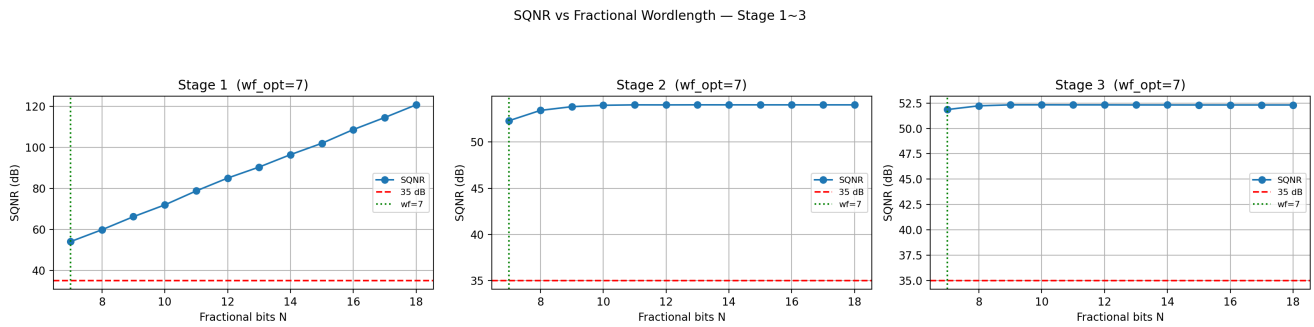


Figure 3.1: Stage 1 - 3 : SQNR vs Fractional Wordlength

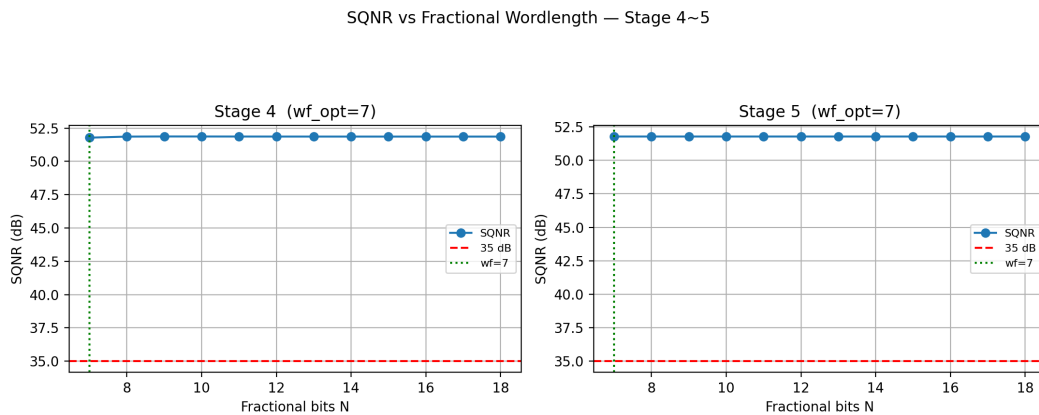


Figure 3.2: Stage 4 - 5 : SQNR vs Fractional Wordlength

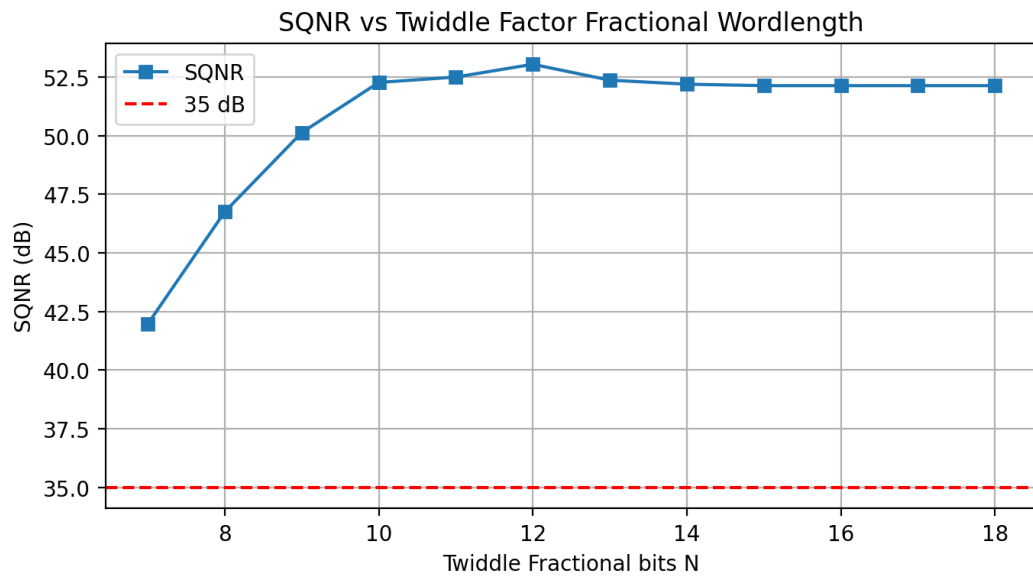


Figure 3.3: SQNR vs Fractional Wordlength of Twiddle Factors

Step 4 — Control Signals: Butterfly and Commutator

4.1 Result 4 - Control Signal Generation

Now, use a 5-bit counter counting from 0 to 31, which is synchronized to the input index. Generate the control signals of commutator and butterfly units for each stage. Show how you generate them and why? (20%)

For a 32-point SDF DIF FFT, each stage i ($i = 1 \dots 5$) has a delay buffer of length $d_i = 32/2^i$. The butterfly unit switches modes every d_i clock cycles:

- **Bypass (0):** current input is stored into the delay buffer; delay buffer content is forwarded to the multiplier.
- **Butterfly (1):** current input and the delay buffer content are processed by the butterfly; upper output (sum) is stored back into the buffer; lower output (difference) is sent to the multiplier.

Because the switching period is $2d_i$ and $d_i = 2^{5-i}$, the control bit that toggles with exactly this period is counter $[5 - i]$:

$$\boxed{\text{Ctr}_i = \text{counter}[5 - i]} \quad (4.1)$$

Table 4.1: Butterfly/Commutator Control Signals for 32-point SDF FFT

Stage	Delay Buffer	Control Signal	Butterfly (1) when	Bypass (0) when
1	Z^{-16}	counter[4]	cnt = 16–31	cnt = 0–15
2	Z^{-8}	counter[3]	cnt = 8–15, 24–31	cnt = 0–7, 16–23
3	Z^{-4}	counter[2]	cnt = 4–7, 12–15, ...	others
4	Z^{-2}	counter[1]	odd groups of 2	even groups of 2
5	Z^{-1}	counter[0]	odd cnt	even cnt

Step 5 — Control Signals: Complex Multipliers

5.1 Result 5 - Multiplier Control Signal Generation

From Table I, derive your control signal to the complex multiplier block to control the multiplication and bypass operation. (1: Multiplication, 0: bypass operation) (10%)

During the **bypass** phase of the PE, the delay buffer outputs the stored difference values ($h_k = x_k - x_{k+N/2}$), which must be multiplied by the twiddle factor W_N^k . During the **butterfly** phase, the PE lower output carries the sum values ($g_k = x_k + x_{k+N/2}$), which feed the even-indexed sub-DFT and require no twiddle multiplication (effectively $\times 1$). Therefore, the multiplier enable is the *complement* of the butterfly control:

$$\text{Mult_en_i} = \overline{\text{Ctr_i}} = \sim \text{counter}[5 - i] \quad (5.1)$$

Table 5.1: Complex Multiplier Control Signals for 32-point SDF FFT

Stage	Butterfly Control (Ctr_i)	Multiplier Enable (Mult_en_i)
1	counter[4]	$\sim$ counter[4]
2	counter[3]	$\sim$ counter[3]
3	counter[2]	$\sim$ counter[2]
4	counter[1]	$\sim$ counter[1]
5	counter[0]	$\sim$ counter[0]

Step 6 — Twiddle Factor ROM Design

6.1 Result 6 - Lookup Table and Quadrant Symmetry

Since sine and cosine functions have symmetry property, for the ROM32 table with 16 phases, we will only save the sine and cosine values in the first quadrant. Please design the lookup table and the way to generate correct result associated with the counter. Note that if you insert pipeline registers, the counter value could be changed accordingly. Draw the block diagram. (10%)

From the table below we can identify two redundancies that allow the ROM to be greatly compressed:

- **We do not need to store all 16 phases.** The entries for $k = 8 \sim 15$ can be reconstructed from those for $k = 0 \sim 7$ by using the half-period symmetry $\cos(\pi - \theta) = -\cos(\theta)$ and $\sin(\pi - \theta) = \sin(\theta)$.
- **We do not need to store the imaginary (sine) values.** By the quarter-period identity $\sin(\theta) = \cos(\pi/2 - \theta)$, every sine value can be looked up directly from the cosine entries that are already stored.

Combining these two observations, the ROM size is reduced from the original **32 stored values** (16 cosines + 16 sines) down to only **8 first-quadrant cosine values**.

Table 6.1: Reference W_{32}^k ($k = 0 \sim 15$)

k	Real Cosine (FP32)	Real Cosine (FXP)	Imag Sine (FP32)	Imag Sine (FXP)
0	1.000	512	0.000	0
1	0.981	502	0.193	99
2	0.923	473	0.389	195
3	0.830	425	0.555	284
4	0.707	362	0.707	362
5	0.555	284	0.830	425
6	0.389	195	0.923	473
7	0.193	99	0.981	502
8	0.000	0	1.000	512
9	-0.195	-99	0.981	502
10	-0.389	-195	0.923	473
11	-0.555	-284	0.830	425
12	-0.707	-362	0.707	362
13	-0.830	-425	0.555	284
14	-0.923	-473	0.389	195
15	-0.981	-502	0.193	99

The figure below shows the **Reduced-ROM32** design built upon the observations discussed previously. The bit counter [3] is used to identify whether the current phase lies in the **first or second quadrant**, which in turn decides whether the looked-up cosine value should be negated. In addition, the boundary cases counter = 0 and counter = 8 are **hard-coded** (as 1 and 0 respectively), so that no extra entry is needed in the lookup table.

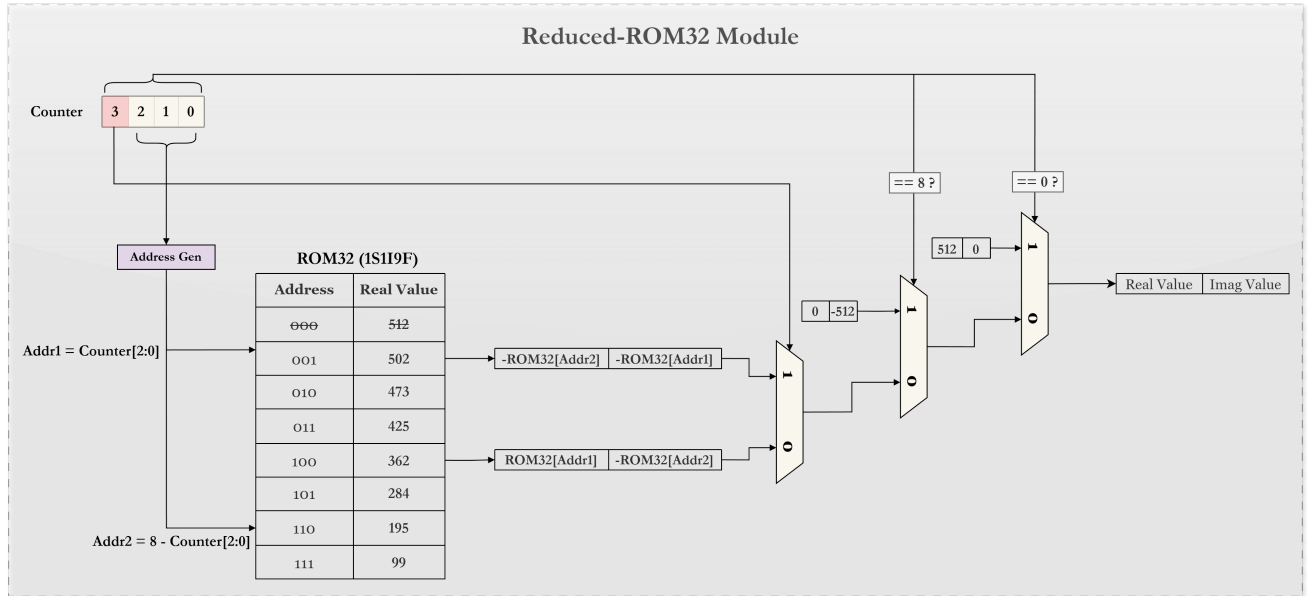


Figure 6.1: Block Diagram of Reduced-ROM32

As shown in the figure below, once **pipeline registers** are inserted between adjacent stages, the **counter must be pipelined together with the datapath** by the same number of delay cycles. Otherwise, the counter value would no longer be aligned with the data flowing through each stage, and the ROM would output the **wrong twiddle factor**.

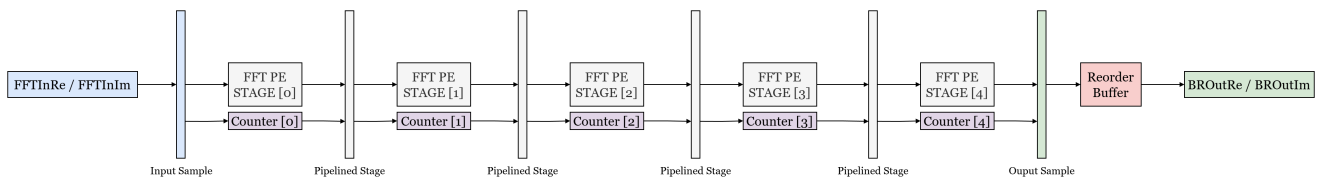


Figure 6.2: Block Diagram of Pipelined FFT

Step 7 — Bit-Reversal Ping-Pong Buffer

7.1 Result 7 - Timing Diagram of Bit-Reversal Module

Show the timing diagram of behavior simulation for bit-reverse re-ordering with ping-pong accessed bit-reversal buffer as in Fig. 8. (15%)

As shown in the timing diagram, I input **two consecutive sets of data into the Bit-Reversal Module along with the InValid signal**. After all the input data has been sent, the InValid signal is pulled low. It can be observed that after **32 cycles, the OutValid signal goes high and the reordered data is output**. The second set of data is also output subsequently. After all the data is output, **the OutValid signal goes low**. In the waveform, the upper part shows the input and output signals, while the lower part displays internal registers or nets.

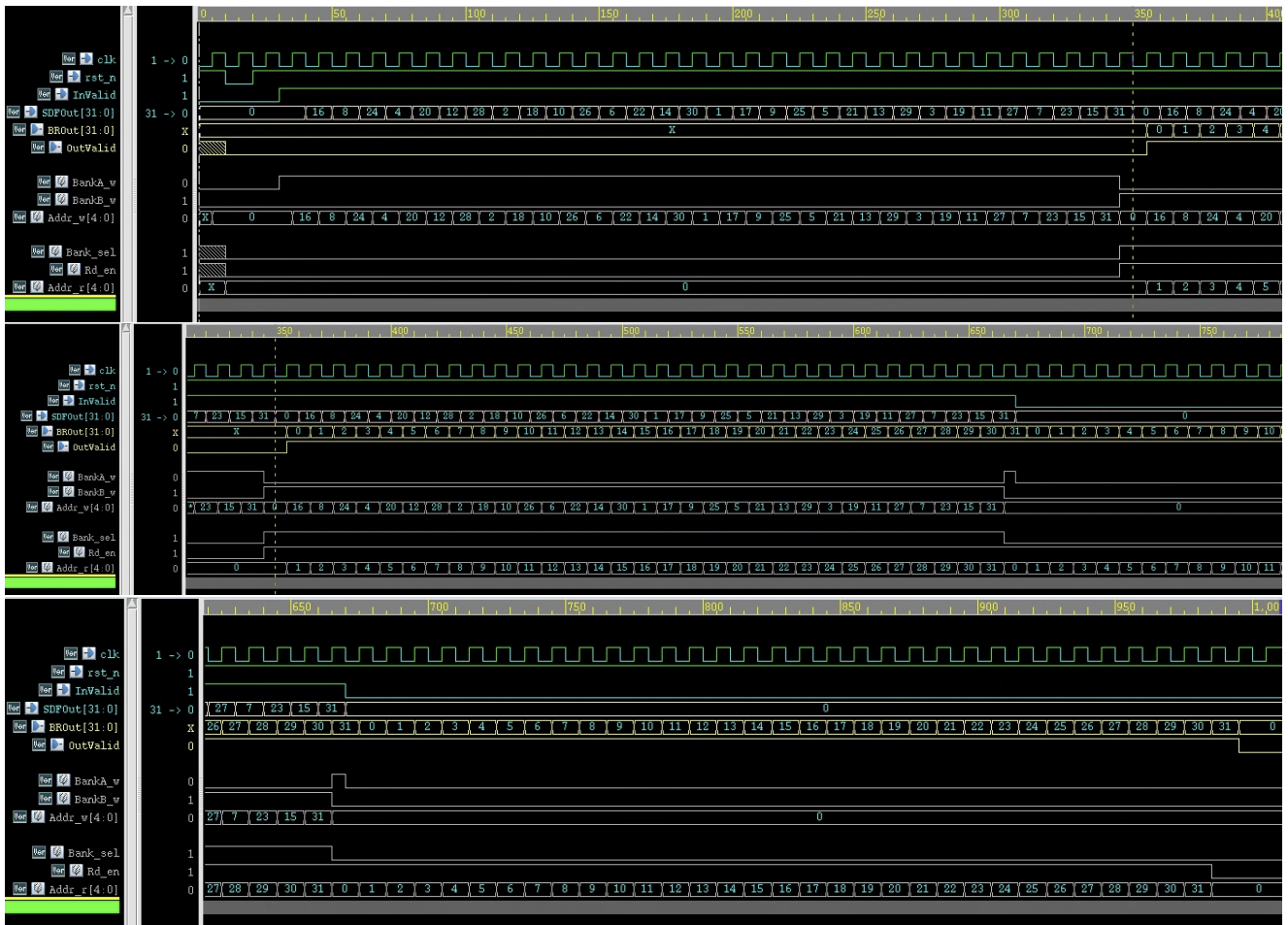


Figure 7.1: Timing Diagram of Bit-Reversal Module

Step 8 — SDF FFT Behavior Simulation

8.1 Result 8 - Single Symbol Simulation

Please use `FFTInput32.mat` as the inputs. Quantize them using the wordlength selected by yourself. Show the timing diagram of behavior simulation for SDF FFT. Compare the results to your answer in 32 Samples (Step 1). Draw the error for 32 samples of real part and imaginary part. (25%)

The fixed-point wordlengths used in the SDF FFT behavior simulation are summarized in the table below, chosen from the Step 3 SQNR analysis ($N = 7$). **Data** signals adopt a **1S 6I 7F** format (14 bits total), while the **twiddle factors** stored in the ROM use a more compact **1S 1I 7F** format (9 bits total) because their magnitudes are bounded by 1.

Table 8.1: Fixed-Point Wordlength Configuration

Signal	Sign	Integer	Fractional	Total
Data	1	6	7	14
Twiddle Factor	1	1	7	9

The SQNR of the 32-sample single-symbol simulation (hardware output vs. Step 1 float SDF) is **35.21** dB.

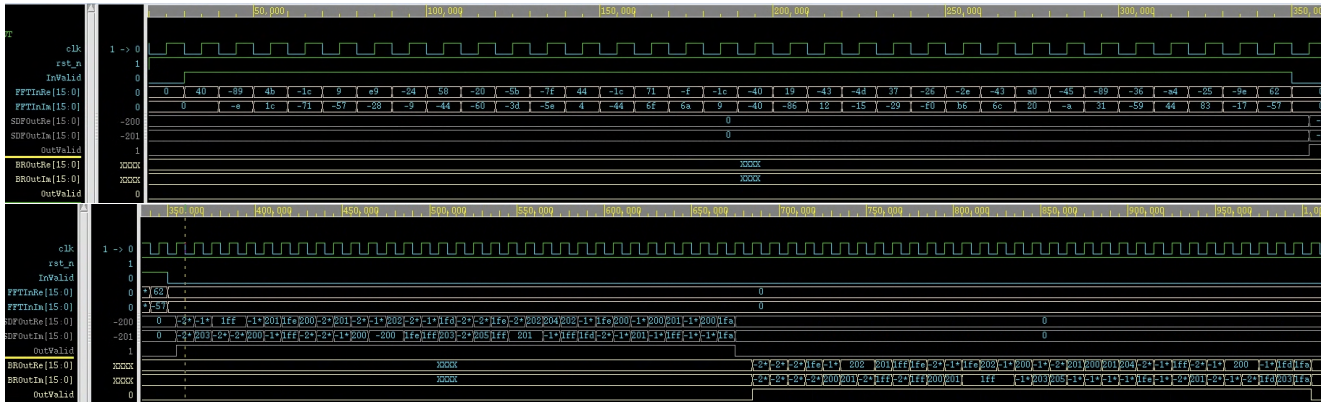


Figure 8.1: Timing Diagram of 32 Samples

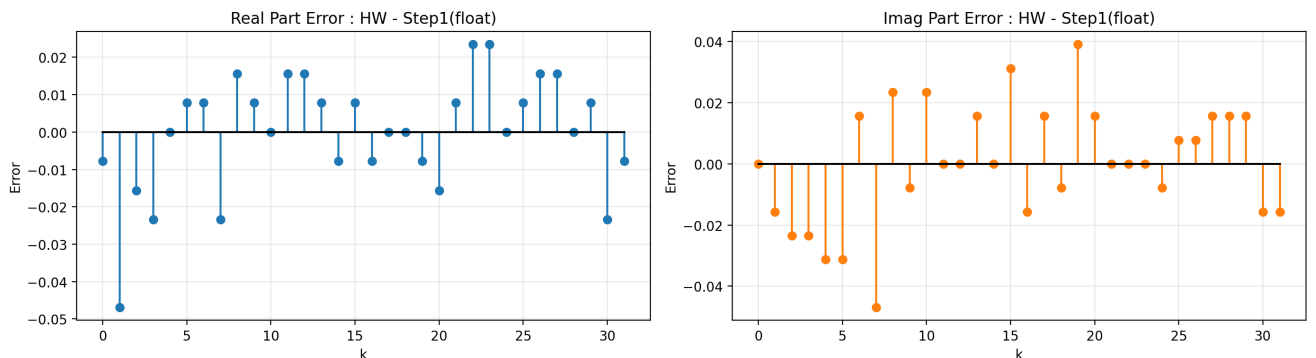


Figure 8.2: Error of 32 Samples

Step 9 — Streaming Simulation (96 Samples)

9.1 Result 9 - Streaming-In Streaming-Out Verification

Use your own inputs of 96 Samples (Step 3). Show the timing diagram of behavior simulation with streaming-input and streaming-output. Draw the error for 96 samples of real part and imaginary part. (25%) Calculate the SQNR of 96 samples.

The SQNR of the 96-sample streaming simulation (hardware output vs. np.fft.golden) is **40.99 dB**.



Figure 9.1: Timing Diagram of 96 Samples

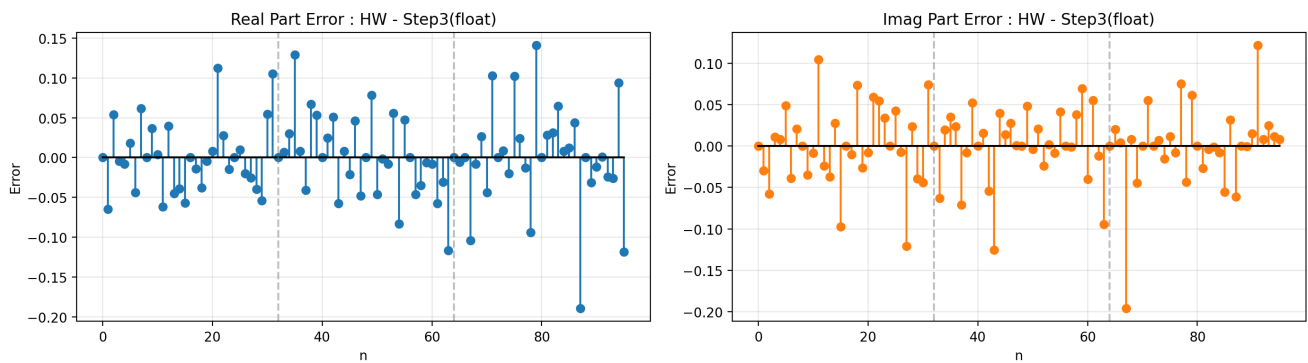


Figure 9.2: Error of 96 Samples

Step 10 — Synthesis Without Pipeline

10.1 Result 10 - Pre-Pipeline Synthesis Report

Insert D flip-flop at the input and output. Synthesize your design. Provide the report of max delay. Note that if you did not insert internal pipeline registers, the critical path is long and the operating frequency is not high. (10%)

The pre-pipeline design is synthesized with Synopsys Design Compiler at an **operating frequency of 250 MHz** (clock period $T_{\text{clk}} = 4.00$ ns). The worst setup path is a register-to-register path from cnt_reg through the combinational SDF FFT datapath to SDFOutRe_reg. Under this constraint, the **critical path delay** (data arrival time) is **2.68** ns, and the corresponding setup slack is **1.71** ns (MET).

Table 10.1: Critical Path Report (Pre-Pipeline)

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.50	0.50
cnt_reg[0]/CP (DFCNQD2BWP20P90)	0.00#	0.50 r
cnt_reg[0]/Q (DFCNQD2BWP20P90)	0.13	0.63 f
...		
U1471/Z (XOR2D1BWP20P90)	0.03	2.68 r
SDFOutRe_reg[15]/D (DFCNQD1BWP20P90)	0.00	2.68 r
data arrival time		2.68
clock clk (rise edge)	4.00	4.00
clock network delay (ideal)	0.50	4.50
clock uncertainty	-0.10	4.40
SDFOutRe_reg[15]/CP (DFCNQD1BWP20P90)	0.00#	4.40 r
library setup time	-0.02	4.38
data required time		4.38
data required time		4.38
data arrival time		-2.68
slack (MET)		1.71

Step 11 — Pipeline Synthesis at 300 MHz

11.1 Result 11 - Post-Pipeline Synthesis and Simulation

Insert pipeline registers to accelerate your design. The target operating frequency (f_s) is 300 MHz. Show the timing diagram of post-synthesis simulation results with proper clock period settings ($1/f_s$). Draw the error for 96 samples of real part and imaginary part. (25%)

As shown in the Figure 6.2, I insert five pipelined stages to accelerate the design.

The clock period is set to $1/300$ MHz and the SQNR of the 96-sample post-synthesis simulation is **40.99** dB.



Figure 11.1: Post-Synthesis Timing Diagram of 96 Samples

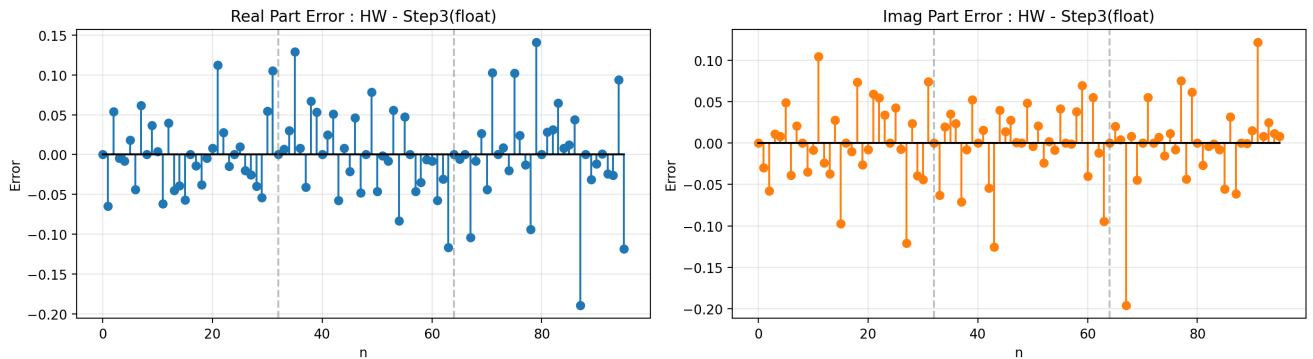


Figure 11.2: Post-Synthesis Error of 96 Samples

The pipelined design is synthesized at the target **operating frequency of 300 MHz** (clock period $T_{\text{clk}} = 3.20$ ns). The reported critical path is a register-to-output path from Reorder_Unit/BR0ut_reg to BR0utIm. The **critical path delay** (data arrival time) is **0.63** ns, with a setup slack of **1.37** ns (MET).

Table 11.1: Critical Path Report (Post-Pipeline, 300 MHz)

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.50	0.50
Reorder_Unit/BR0ut_reg[0]/CP (DFCNQD1BWP20P90)	0.00#	0.50 r
Reorder_Unit/BR0ut_reg[0]/Q (DFCNQD1BWP20P90)	0.13	0.63 r
Reorder_Unit/BR0ut_reg[0] (Reorder_Buffer)	0.00	0.63 r
BR0utIm[0] (out)	0.00	0.63 r
data arrival time		0.63
clock clk (rise edge)	3.20	3.20
clock network delay (ideal)	0.50	3.70
clock uncertainty	-0.10	3.60
output external delay	-1.60	2.00
data required time		2.00
data required time		2.00
data arrival time		-0.63
slack (MET)		1.37